

Brewing the Soundness of BEer

Denotational Semantics, Set Theory and B

Vincent Trélat

Université de Lorraine, CNRS, INRIA, Loria, Nancy, France



- 1 Introduction
- 2 Encoding B POs in SMT-LIB using HO
- 3 Results
- 4 Denotational semantics of B
- 5 Soundness of the encoding

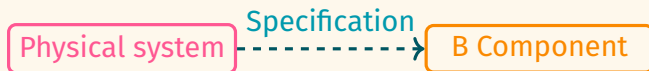
Introduction

B method

Physical system

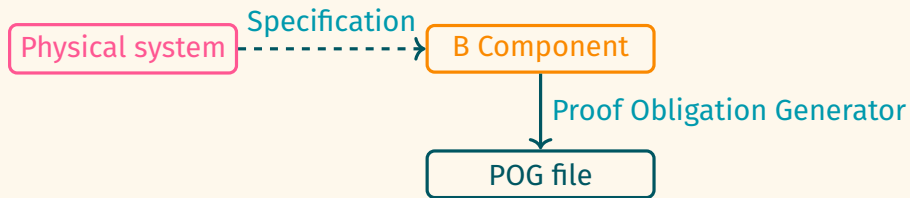
Introduction

B method



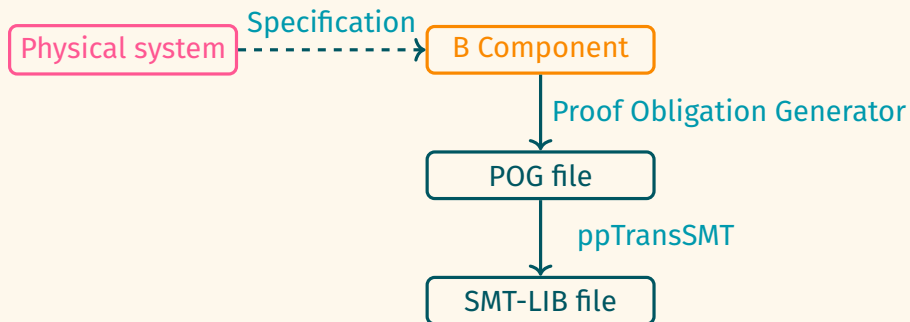
Introduction

B method



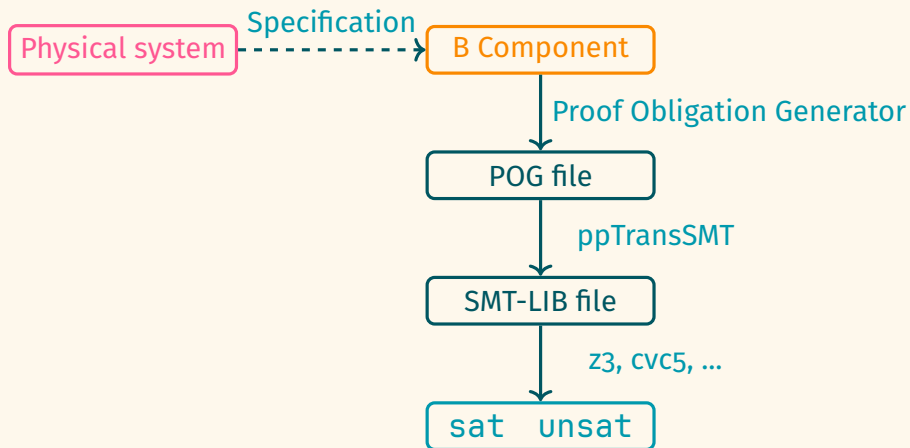
Introduction

B method



Introduction

B method



Introduction

SMT-LIB (up to v2.6)

- Standard input format for SMT solvers (e.g. **z3**, **cvc5**, **veriT**)
- Based on many-sorted **first-order logic**
- Comes with many **theories** (e.g. arrays, integer and real arithmetic)

Introduction

SMT-LIB (up to v2.6)

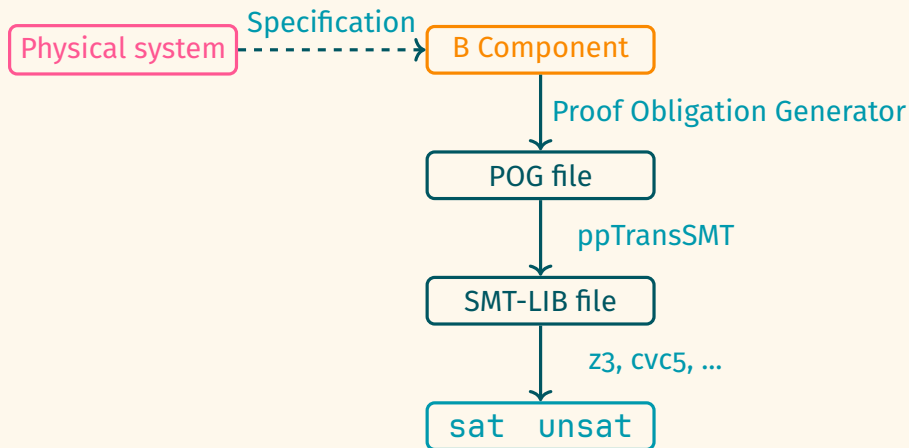
- Standard input format for SMT solvers (e.g. **z3**, **cvc5**, **veriT**)
- Based on many-sorted **first-order logic**
- Comes with many **theories** (e.g. arrays, integer and real arithmetic)

SMT-LIB v2.7

- Brings **higher-order constructs** through λ -abstractions
- Brings **higher-order types** through arrow type constructor
- Only supported by **cvc5** yet

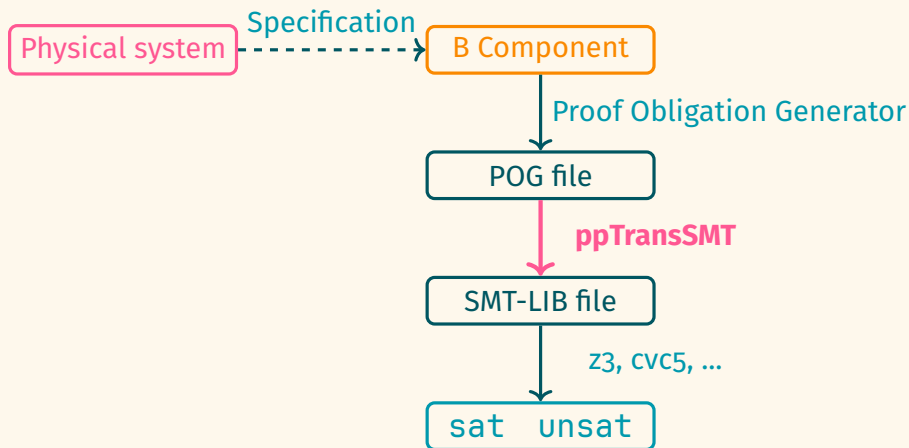
Introduction

B method



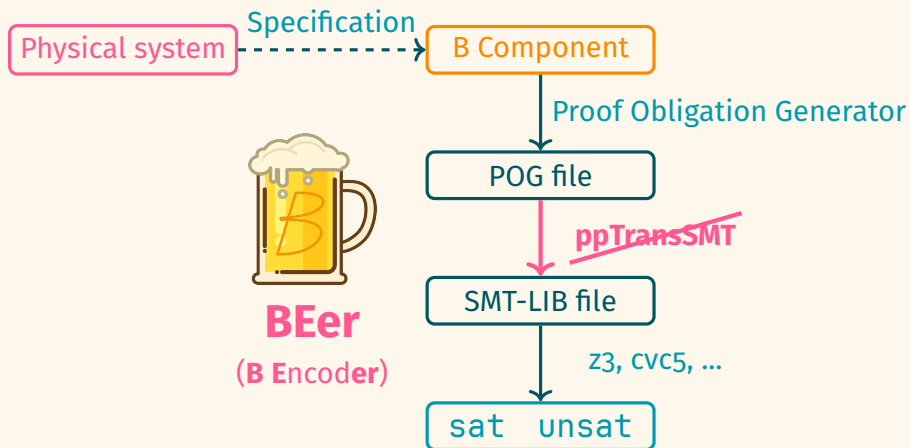
Introduction

B method



Introduction

B method



Introduction

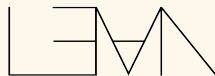
Because Germany taught me to lean on beer,

Introduction

Because Germany taught me to lean on beer,



is written in



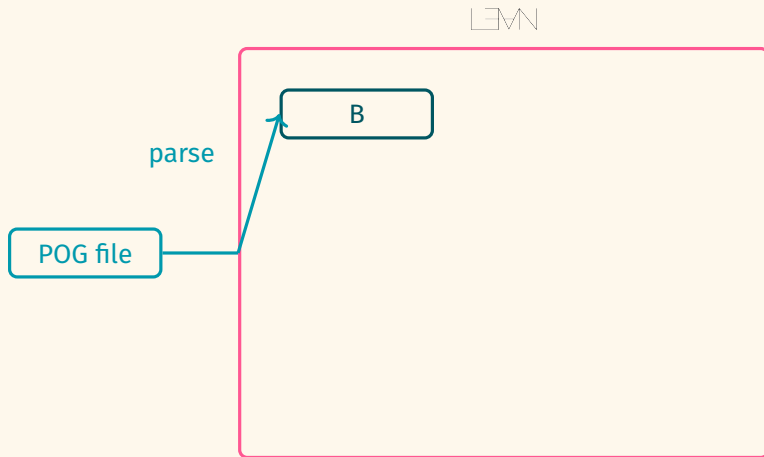
Architecture of

Architecture of 🍺

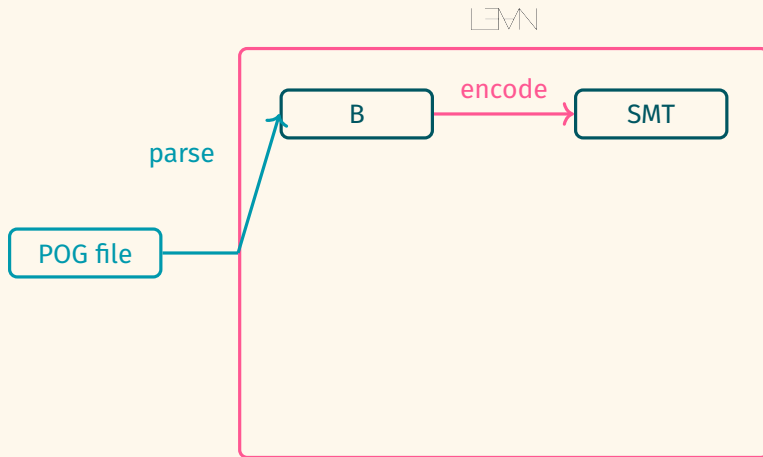
LEVN

POG file

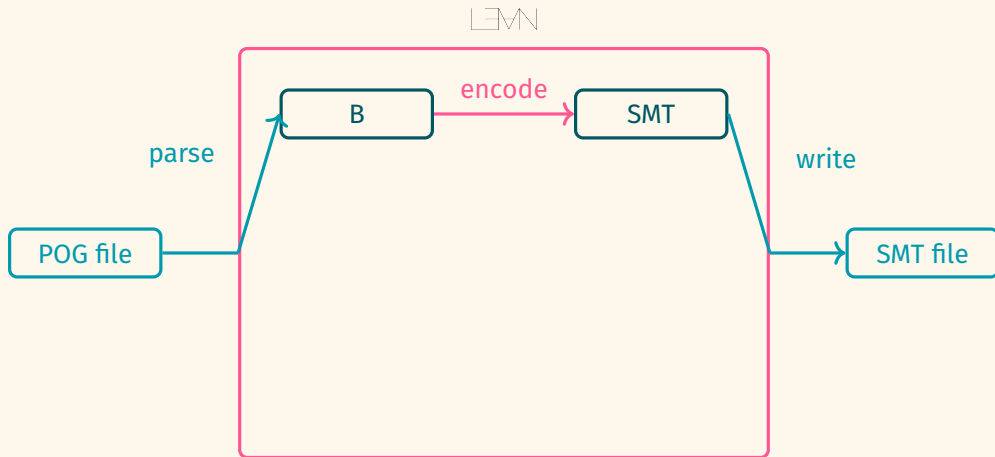
Architecture of 🍺



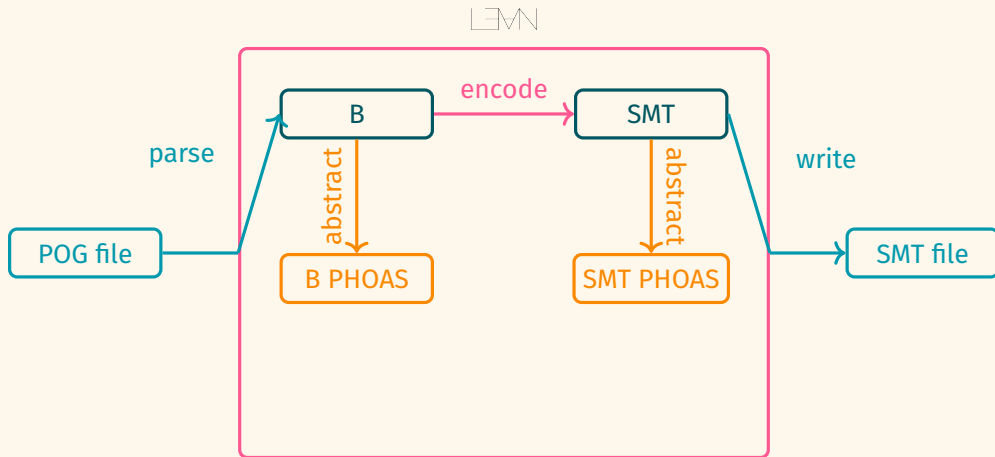
Architecture of



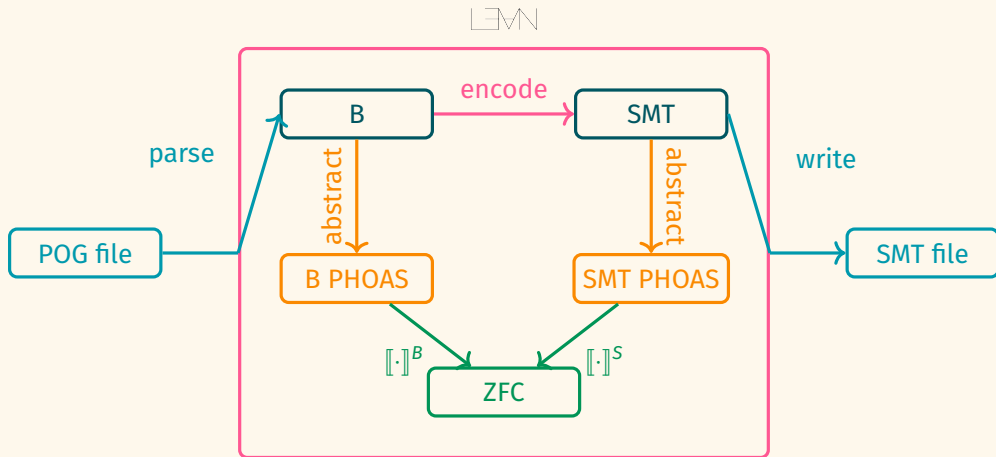
Architecture of



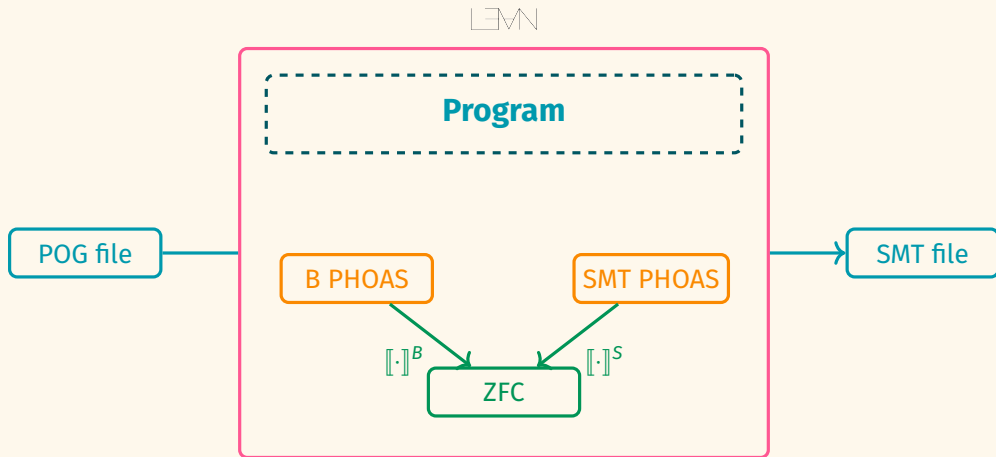
Architecture of



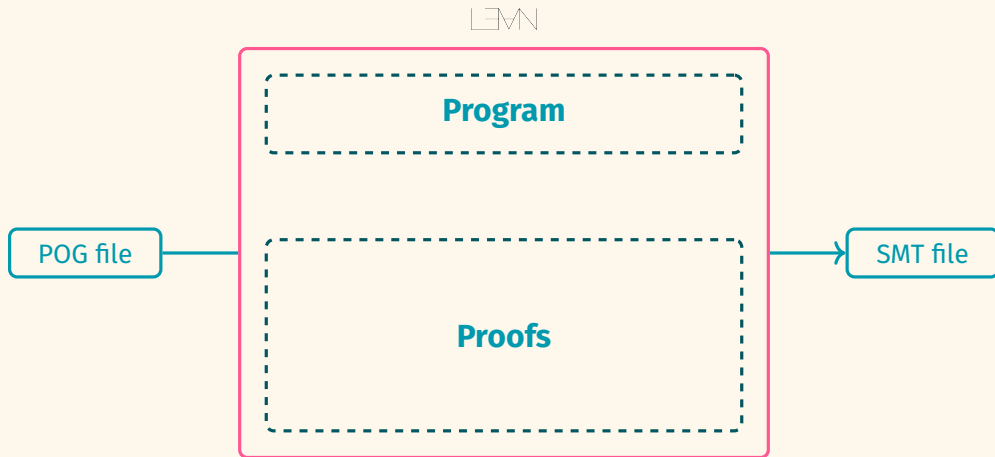
Architecture of



Architecture of



Architecture of



- 1 Introduction
- 2 **Encoding B POs in SMT-LIB using HO**
 - Overview of the encoding
- 3 Results
- 4 Denotational semantics of B
- 5 Soundness of the encoding

Overview of the encoding

(former) first-order encoding



(new) higher-order encoding 🍺



Overview of the encoding

(former) first-order encoding

- FOL

(new) higher-order encoding 🍺

- HOL

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, P and C

(new) higher-order encoding

- HOL
- Definition of sets via characteristic predicates

Overview of the encoding

SETS

$S = \{e1, e2, e3\}$

Overview of the encoding

SETS

$S = \{e1, e2, e3\}$

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-fun S () (P Int))
(declare-fun e1 () Int)
(declare-fun e2 () Int)
(declare-fun e3 () Int)
(assert (distinct e1 e2 e3))
(declare-fun  $\in_0$  ((Int) (P Int)) Bool)

(assert (forall ((x Int)) (=
  ( $\in_0$  x S)
  (or (= x e1) (= x e2) (= x e3)))))
```

Overview of the encoding

SETS

$S = \{e1, e2, e3\}$

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-fun S () (P Int))
(declare-fun e1 () Int)
(declare-fun e2 () Int)
(declare-fun e3 () Int)
(assert (distinct e1 e2 e3))
(declare-fun  $\in_0$  ((Int) (P Int)) Bool)

(assert (forall ((x Int)) (=
  ( $\in_0$  x S)
  (or (= x e1) (= x e2) (= x e3)))))
```

(new) higher-order encoding

```
(declare-const e1 Int)
(declare-const e2 Int)
(declare-const e3 Int)
(assert (distinct e1 e2 e3))
(define-const S ( $\rightarrow$  Int Bool)
  (lambda ((x Int))
    (or (= x e1) (= x e2) (= x e3))))
```

Overview of the encoding

(former) first-order encoding

- FOL
- Specification of sets via $\in$, P and C

(new) higher-order encoding

- HOL
- Definition of sets via characteristic predicates

Overview of the encoding

(former) first-order encoding

- FOL
- **Specification** of sets via $\in$, P and C
- Only expressions like $x \in S$ are encoded

(new) higher-order encoding

- HOL
- **Definition** of sets via characteristic predicates
- Sets alone make sense; $x \in S$ is true by definition

Overview of the encoding

(former) first-order encoding

- FOL
- **Specification** of sets via $\in$, P and C
- Only expressions like $x \in S$ are encoded
- Functions are **functional relations**

(new) higher-order encoding 🍺

- HOL
- **Definition** of sets via **characteristic predicates**
- Sets alone make sense; $x \in S$ is true by definition
- Functions are (sometimes) **functions**

Suppose we have a function $f \in A \rightarrow B$.

Suppose we have a function $f \in A \rightarrow B$.

(former) first-order encoding

f is a relation between A and B :

$$f \subseteq A \times B$$

f is functional:

$$\forall x y z, x \mapsto y \in f \wedge x \mapsto z \in f \\ \Rightarrow y = z$$

Suppose we have a function $f \in A \rightarrow B$.

(former) first-order encoding

f is a relation between A and B :

$$f \subseteq A \times B$$

f is functional:

$$\forall x y z, x \mapsto y \in f \wedge x \mapsto z \in f \\ \Rightarrow y = z$$

(new) higher-order encoding 🍺

f is a total function from A to $B \cup \{\star\}$:

$$f \in (B \cup \{\star\})^A$$

Suppose we have a function $f \in A \rightarrow B$.

(former) first-order encoding

f is a relation between A and B :

$$f \subseteq A \times B$$

f is functional:

$$\forall x y z, x \mapsto y \in f \wedge x \mapsto z \in f \\ \Rightarrow y = z$$

(new) higher-order encoding 🍺

f is a total function from A to $B \cup \{\star\}$:

$$f \in (B \cup \{\star\})^A$$

```
(declare-datatype Option  
  (par (T) ((some (the T)) (none)))))
```

Suppose we have a function $f \in A \rightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

Suppose we have a function $f \in A \rightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C  $\tau_A$   $\tau_B$ )))
(declare-fun
   $\in_0$  ( $\tau_A$   $\tau_B$  (P (C  $\tau_A$   $\tau_B$ ))) Bool)
(assert
  (forall ((x  $\tau_A$ ) (y  $\tau_B$ ) (z  $\tau_B$ ))
    ( $\Rightarrow$  (and ( $\in_0$  x y f) ( $\in_0$  x z f))
      (= y z))))
```

Suppose we have a function $f \in A \rightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C  $\tau_A$   $\tau_B$ )))
(declare-fun
   $\in_0$  ( $\tau_A$   $\tau_B$  (P (C  $\tau_A$   $\tau_B$ ))) Bool)
(assert
  (forall ((x  $\tau_A$ ) (y  $\tau_B$ ) (z  $\tau_B$ ))
    ( $\Rightarrow$  (and ( $\in_0$  x y f) ( $\in_0$  x z f))
      (= y z))))
```

(new) higher-order encoding

```
(declare-const f ( $\rightarrow$   $\tau_A$  (Option  $\tau_B$ )))
```

Suppose we have a function $f \in A \rightarrow B$. Let τ_A and τ_B represent the types of A and B respectively.

(former) first-order encoding

```
(declare-sort P 1)
(declare-sort C 2)
(declare-const f (P (C  $\tau_A$   $\tau_B$ )))
(declare-fun
   $\in_0$  ( $\tau_A$   $\tau_B$  (P (C  $\tau_A$   $\tau_B$ ))) Bool)
(assert
  (forall ((x  $\tau_A$ ) (y  $\tau_B$ ) (z  $\tau_B$ ))
    ( $\Rightarrow$  (and ( $\in_0$  x y f) ( $\in_0$  x z f))
      (= y z))))
```

(new) higher-order encoding

```
(declare-const f ( $\rightarrow$   $\tau_A$  (Option  $\tau_B$ )))
```

+ specification that **dom** $f \subseteq A$ and **ran** $f \subseteq B$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int} \cdot \exists b: \text{int}, f: \text{set}(\tau \times \text{int}) \cdot f \in S \mapsto a..b$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int} \cdot \exists b: \text{int}, f: \text{set}(\tau \times \text{int}) \cdot f \in S \rightarrow a..b$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int} \cdot \exists b: \text{int}, f: \text{set}(\tau \times \text{int}) \cdot f \in S \rightarrow a..b \wedge S \subseteq \text{dom}(f) \wedge \text{inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int} \cdot \exists b: \text{int}, f: \text{set}(\tau \times \text{int}) \cdot f \in S \rightarrow a..b \wedge S \subseteq \text{dom}(f) \wedge \text{inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$f \in S \leftrightarrow a..b$$
 $\wedge$
$$\text{_func}(f)$$
 $\wedge$
$$S \subseteq \text{_dom}(f)$$
 $\wedge$
$$\text{_inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$f \in S \leftrightarrow a..b$$
 $\wedge$
$$\text{\color{red}_func}(f)$$
 $\wedge$
$$S \subseteq \text{dom}(f)$$
 $\wedge$
$$\text{_inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$f \in S \leftrightarrow a..b$$
 $\wedge$
$$\forall x: \tau, y: \text{int}, z: \text{int}. x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z$$
 $\wedge$
$$S \subseteq \mathbf{dom}(f)$$
 $\wedge$
$$_ \text{inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$f \in S \leftrightarrow a..b$$
 $\wedge$
$$\forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z$$
 $\wedge$
$$S \subseteq \text{dom}(f)$$
 $\wedge$
$$\neg \text{inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$\forall x: \tau, y: \text{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \quad \wedge$$
$$\forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \quad \wedge$$
$$S \subseteq \text{dom}(f) \quad \wedge$$
$$\neg \text{inj}(f)$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\begin{aligned} & \forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}). \\ & \quad \forall x: \tau, y: \text{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \quad \wedge \\ & \quad \forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \quad \wedge \\ & \quad S \subseteq \text{dom}(f) \quad \wedge \\ & \quad \neg \text{inj}(f) \end{aligned}$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\begin{aligned} & \forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}). \\ & \quad \forall x: \tau, y: \text{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \quad \wedge \\ & \quad \forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \quad \wedge \\ & \quad \forall z: \tau \cdot z \in S \Rightarrow \exists w: \text{int} \cdot z \mapsto w \in f \quad \wedge \\ & \quad \text{_inj}(f) \end{aligned}$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\begin{aligned} \forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}). & \\ \quad \forall x: \tau, y: \text{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b & \quad \wedge \\ \quad \forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z & \quad \wedge \\ \quad \forall z: \tau \cdot z \in S \Rightarrow \exists w: \text{int} \cdot z \mapsto w \in f & \quad \wedge \\ \text{_inj}(f) & \end{aligned}$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S is defined as follows in B:

$$\forall a: \text{int}. \exists b: \text{int}, f: \text{set}(\tau \times \text{int}).$$
$$\forall x: \tau, y: \text{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \quad \wedge$$
$$\forall x: \tau, y: \text{int}, z: \text{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \quad \wedge$$
$$\forall z: \tau \cdot z \in S \Rightarrow \exists w: \text{int} \cdot z \mapsto w \in f \quad \wedge$$
$$\forall x: \tau, y: \tau, z: \tau \cdot x \mapsto z \in f \wedge y \mapsto z \in f \Rightarrow x = y$$

How large is the gain?

Let S be a set of elements of type τ .

The expression **finite** S can be encoded as follows:

$$\exists N: \text{int}, f: \tau \rightarrow \text{int}.$$
$$\forall x: \tau, y: \tau, z: \text{int} \cdot f(x) = z \wedge f(y) = z \Rightarrow x = y \quad \wedge$$
$$\forall x: \tau \cdot x \in S \Rightarrow 0 \leq f(x) \wedge f(x) < N$$

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 : \in \mathcal{P}(\text{NAT})$

END

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 : \in \mathcal{P}(\text{NAT})$

END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \subseteq \text{NAT} \wedge s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$$

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 := \mathcal{P}(\text{NAT})$

END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \subseteq \text{NAT} \wedge s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$$

which boils down to proving:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \in \text{FIN}(\mathbb{Z})$$

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 : \in \mathcal{P}(\text{NAT})$

END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \subseteq \text{NAT} \wedge s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$$

which boils down to proving:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \in \text{FIN}(\mathbb{Z})$$

X predicate prover from Atelier B

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 : \in \mathcal{P}(\text{NAT})$

END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \subseteq \text{NAT} \wedge s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$$

which boils down to proving:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \in \text{FIN}(\mathbb{Z})$$

✗ predicate prover from Atelier B

✗ CVC5 with ppTransSMT

Does this work?

MACHINE

M

VARIABLES

s0

INVARIANT

$s0 \subseteq \text{NAT} \wedge$

$s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$

INITIALISATION

$s0 : \in \mathcal{P}(\text{NAT})$

END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \subseteq \text{NAT} \wedge s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \text{FIN}(\mathbb{Z})$$

which boils down to proving:

$$s0 \in \mathcal{P}(\text{NAT}) \Rightarrow s0 \in \text{FIN}(\mathbb{Z})$$

✗ predicate prover from Atelier B

✗ CVC5 with ppTransSMT

✓ CVC5 with 

- 1 Introduction
- 2 Encoding B POs in SMT-LIB using HO
- 3 Results**
- 4 Denotational semantics of B
- 5 Soundness of the encoding

Results

In the current state of  we have:

ppTrans \ 	unsat	sat	unknown	Total
	unsat	sat	unknown	Total
unsat	14,831	0	1,062	15,893
sat	0	0	0	0
unknown	272	0	780	1,052
Total	15,103	0	1,842	16,945

Benchmark specs:

- **681,285** POs in total
- Apple M2 (10 CPU cores, 24 GB RAM)
- cvc5 with incremental mode, MBQI enabled and 3s timeout per query

Results

In the current state of  we have:

 / ppTrans	unsat	sat	unknown	Total
unsat	14,831	0	1,062	15,893
sat	0	0	0	0
unknown	272	0	780	1,052
Total	15,103	0	1,842	16,945

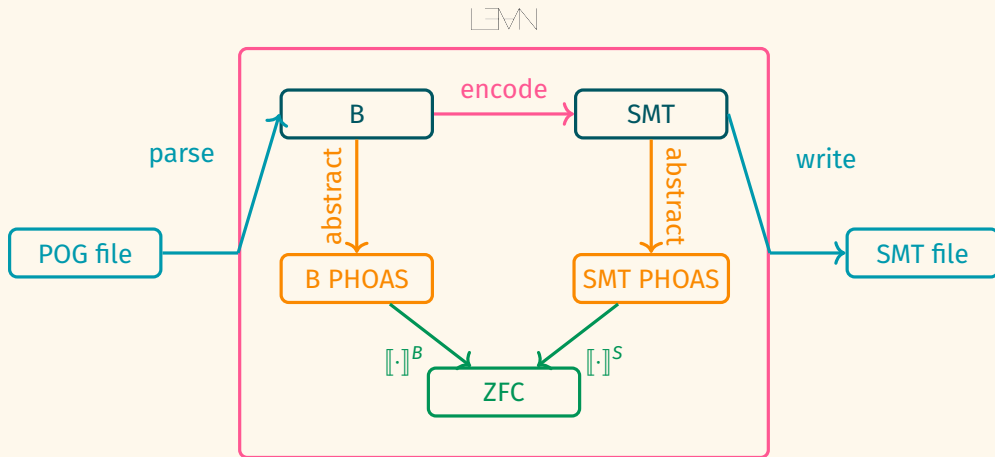
Benchmark specs:

- **681,285** POs in total
- Apple M2 (10 CPU cores, 24 GB RAM)
- cvc5 with incremental mode, MBQI enabled and 3s timeout per query

Conclusion: not that good on “simple” POs but **performs well on HO POs!**

- 1 Introduction
- 2 Encoding B POs in SMT-LIB using HO
- 3 Results
- 4 Denotational semantics of B**
 - Definitions
 - Reasoning about B terms
- 5 Soundness of the encoding

Architecture of



Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory:

Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory: Lean provides a lacunary model of ZFC in Mathlib that we first had to extend.

Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory: Lean provides a lacunary model of ZFC in Mathlib that we first had to extend.

- ZFSet
- naturals, $\mathbb{N}$, ZFNat and arithmetics
- integers, $\mathbb{Z}$, ZFInt and arithmetics
- rationals, $\mathbb{Q}$, ZFRat and arithmetics
- reals, $\mathbb{R}$, ZFReal and arithmetics

Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory: Lean provides a lacunary model of ZFC in Mathlib that we first had to extend.

- ZFSet
- naturals, $\mathbb{N}$, ZFNat and arithmetics
- integers, $\mathbb{Z}$, ZFInt and arithmetics
- rationals, $\mathbb{Q}$, ZFRat and arithmetics
- reals, $\mathbb{R}$, ZFReal and arithmetics

Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory: Lean provides a lacunary model of ZFC in Mathlib that we first had to extend.

- ZFSet
- naturals, $\mathbb{N}$, ZFNat and arithmetics
- integers, $\mathbb{Z}$, ZFInt and arithmetics
- rationals, $\mathbb{Q}$, ZFRat and arithmetics
- reals, $\mathbb{R}$, ZFReal and arithmetics

Denotational semantics of B

ZFC

Interpreting the B syntax in ZFC set theory: Lean provides a lacunary model of ZFC in Mathlib that we first had to extend.

- ZFSet
- naturals, $\mathbb{N}$, ZFNat and arithmetics
- integers, $\mathbb{Z}$, ZFInt and arithmetics
- rationals, $\mathbb{Q}$, ZFRat and arithmetics
- reals, $\mathbb{R}$, ZFReal and arithmetics

Denotational semantics of B

We define a denotation function for abstract B terms:

$$\llbracket \cdot \rrbracket^B : \text{Term } \mathcal{V} \rightarrow \mathcal{V}$$

Denotational semantics of B

We define a denotation function for abstract B terms:

$$\llbracket \cdot \rrbracket^B : \text{Term ZFSet} \rightarrow \text{ZFSet}$$

Denotational semantics of B

We define a denotation function for abstract B terms:

$$\llbracket \cdot \rrbracket^B : \text{Term Dom} \rightarrow \text{Dom}$$

Denotational semantics of B

We define a **denotation function** for abstract B terms:

$$\llbracket \cdot \rrbracket^B : \text{Term Dom} \rightarrow \text{Dom}$$

where

$$\text{Dom} := \sum_{x, \tau} x \in \llbracket \tau \rrbracket^Z$$
$$\begin{cases} \llbracket \text{int} \rrbracket^Z & := \mathbb{Z}^Z \\ \llbracket \text{bool} \rrbracket^Z & := \mathbb{B}^Z \\ \llbracket \text{set } \alpha \rrbracket^Z & := \mathcal{P}^Z(\llbracket \alpha \rrbracket^Z) \\ \llbracket \alpha \times^B \beta \rrbracket^Z & := \llbracket \alpha \rrbracket^Z \times^Z \llbracket \beta \rrbracket^Z \end{cases}$$

Denotational semantics of B

The whole purpose of the PHOAS is to pour semantics into the syntax:

Denotational semantics of B

The whole purpose of the PHOAS is to pour semantics into the syntax:

$$\llbracket \text{var } v \rrbracket^B := v$$

Denotational semantics of B

The whole purpose of the PHOAS is to pour semantics into the syntax:

$$\llbracket \text{var } \langle V, \tau, \text{pf}_V^\tau \rangle \rrbracket^B := \langle V, \tau, \text{pf}_V^\tau \rangle \quad \text{where } \text{pf}_V^\tau : V \in \llbracket \tau \rrbracket^Z$$

Denotational semantics of B

The whole purpose of the PHOAS is to pour semantics into the syntax:

$$\begin{aligned} \llbracket \text{var } \langle V, \tau, \text{pf}_V^\tau \rangle \rrbracket^B &:= \langle V, \tau, \text{pf}_V^\tau \rangle & \text{where } \text{pf}_V^\tau : V \in \llbracket \tau \rrbracket^Z \\ \llbracket \text{int } n \rrbracket^B &:= \langle \text{ofInt } n, \text{int}, \text{pf}_n \rangle & \text{where } \text{pf}_n : \text{ofInt } n \in \mathbb{Z}^Z \end{aligned}$$

Denotational semantics of B

The whole purpose of the PHOAS is to pour semantics into the syntax:

$$\begin{aligned} \llbracket \text{var } \langle V, \tau, \text{pf}_V^\tau \rangle \rrbracket^B &:= \langle V, \tau, \text{pf}_V^\tau \rangle & \text{where } \text{pf}_V^\tau: V \in \llbracket \tau \rrbracket^Z \\ \llbracket \text{int } n \rrbracket^B &:= \langle \text{ofInt } n, \text{int}, \text{pf}_n \rangle & \text{where } \text{pf}_n: \text{ofInt } n \in \mathbb{Z}^Z \\ \llbracket \text{bool } b \rrbracket^B &:= \langle \text{ofBool } b, \text{bool}, \text{pf}_b \rangle & \text{where } \text{pf}_b: \text{ofBool } b \in \mathbb{B}^Z \end{aligned}$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar.

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

The denotation is then constructed as follows:

$$\llbracket x \mapsto^B y \rrbracket^B := \langle (X, Y)^Z, \alpha \times^B \beta, \text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \rangle$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

The denotation is then constructed as follows:

$$\llbracket x \mapsto^B y \rrbracket^B := \langle (X, Y)^Z, \alpha \times^B \beta, \text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \rangle$$

where

$$\text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} : (X, Y)^Z \in \llbracket \alpha \times^B \beta \rrbracket^Z$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

The denotation is then constructed as follows:

$$\llbracket x \mapsto^B y \rrbracket^B := \langle (X, Y)^Z, \alpha \times^B \beta, \text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \rangle$$

where

$$\text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \dashv (X, Y)^Z \in \llbracket \alpha \rrbracket^Z \times^Z \llbracket \beta \rrbracket^Z$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

The denotation is then constructed as follows:

$$\llbracket x \mapsto^B y \rrbracket^B := \langle (X, Y)^Z, \alpha \times^B \beta, \text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \rangle$$

where

$$\text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \dashv X \in \llbracket \alpha \rrbracket^Z \wedge Y \in \llbracket \beta \rrbracket^Z$$

Denotational semantics of B

Simple cases

Let's consider $\mapsto^B$ only; other binary cases are similar. Let x and y be two abstract terms such that:

$$\llbracket x \rrbracket^B = \langle X, \alpha, \text{pf}_X^\alpha \rangle \quad \text{and} \quad \llbracket y \rrbracket^B = \langle Y, \beta, \text{pf}_Y^\beta \rangle$$

The denotation is then constructed as follows:

$$\llbracket x \mapsto^B y \rrbracket^B := \langle (X, Y)^Z, \alpha \times^B \beta, \text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \rangle$$

where

$$\text{pf}_{(X,Y)^Z}^{\alpha \times^B \beta} \dashv \underbrace{X \in \llbracket \alpha \rrbracket^Z}_{\text{pf}_X^\alpha} \wedge \underbrace{Y \in \llbracket \beta \rrbracket^Z}_{\text{pf}_Y^\beta}$$

Denotational semantics of B

Forall quantifier

Binders are a little more tedious.

Denotational semantics of B

Forall quantifier

Binders are a little more tedious. Consider $D := \{e_1, \dots, e_n\}$. Intuition:

$$\llbracket \forall^B D \cdot P \rrbracket^B = \llbracket P \ e_1 \rrbracket^B \wedge^Z \dots \wedge^Z \llbracket P \ e_n \rrbracket^B$$

Denotational semantics of B

Forall quantifier

Binders are a little more tedious. Consider $D := \{e_1, \dots, e_n\}$. Intuition:

$$\llbracket \forall^B D \cdot P \rrbracket^B = \llbracket P \ e_1 \rrbracket^B \wedge^z \dots \wedge^z \llbracket P \ e_n \rrbracket^B = \bigwedge_{i=1}^n \llbracket P \ e_i \rrbracket^B$$

Denotational semantics of B

Forall quantifier

Binders are a little more tedious. Consider $D := \{e_1, \dots, e_n\}$. Intuition:

$$\begin{aligned} \llbracket \forall^B D \cdot P \rrbracket^B &= \llbracket P \ e_1 \rrbracket^B \wedge^z \dots \wedge^z \llbracket P \ e_n \rrbracket^B = \bigwedge_{i=1}^n \llbracket P \ e_i \rrbracket^B \\ &= \bigwedge_{x \in D} \llbracket P \ x \rrbracket^B \end{aligned}$$

Denotational semantics of B

Forall quantifier

Let $D: \text{Term} \rightarrow \text{Dom}$ and $P: (\text{Fin } n \rightarrow \text{Dom}) \rightarrow \text{Term} \rightarrow \text{Dom}$.

Denotational semantics of B

Forall quantifier

Let $D: \text{Term} \rightarrow \text{Dom}$ and $P: (\text{Fin } n \rightarrow \text{Dom}) \rightarrow \text{Term} \rightarrow \text{Dom}$. Assume that

$$\llbracket D \rrbracket^B = \langle \mathcal{D}, \text{set } \tau, \text{pf}_{\mathcal{D}}^{\text{set } \tau} \rangle \quad \text{and} \quad \tau = \tau_1 \times^B \dots \times^B \tau_n$$

Denotational semantics of B

Forall quantifier

Let $D: \text{Term} \rightarrow \text{Dom}$ and $P: (\text{Fin } n \rightarrow \text{Dom}) \rightarrow \text{Term} \rightarrow \text{Dom}$. Assume that

$$\llbracket D \rrbracket^B = \langle \mathcal{D}, \text{set } \tau, \text{pf}_{\mathcal{D}}^{\text{set } \tau} \rangle \quad \text{and} \quad \tau = \tau_1 \times^B \dots \times^B \tau_n$$

We define the following function, for any ZFC set z :

$$\mathcal{P}(z) := \begin{cases} P_z & \text{if } \exists (x_i)_{1 \leq i \leq n}, \begin{cases} z = (x_1, \dots, x_n)^z \wedge z \in \llbracket \tau \rrbracket^z \\ \llbracket P(\text{var } \langle x_i, \tau_i, \text{pf}_{x_i}^{\tau_i} \rangle)_{1 \leq i \leq n} \rrbracket^B = \langle P_z, -, - \rangle \end{cases} \\ \perp^z & \text{otherwise} \end{cases}$$

Denotational semantics of B

Forall quantifier

Let $D: \text{Term} \rightarrow \text{Dom}$ and $P: (\text{Fin } n \rightarrow \text{Dom}) \rightarrow \text{Term} \rightarrow \text{Dom}$. Assume that

$$\llbracket D \rrbracket^B = \langle \mathcal{D}, \text{set } \tau, \text{pf}_{\mathcal{D}}^{\text{set } \tau} \rangle \quad \text{and} \quad \tau = \tau_1 \times^B \dots \times^B \tau_n$$

We define the following function, for any ZFC set z :

$$\mathcal{P}(z) := \begin{cases} P_z & \text{if } \exists (x_i)_{1 \leq i \leq n}, \begin{cases} z = (x_1, \dots, x_n)^z \wedge z \in \llbracket \tau \rrbracket^z \\ \llbracket P \text{ (var } \langle x_i, \tau_i, \text{pf}_{x_i}^{\tau_i} \rangle)_{1 \leq i \leq n} \rrbracket^B = \langle P_z, -, - \rangle \end{cases} \\ \perp^z & \text{otherwise} \end{cases}$$

At last, we define the following denotation:

$$\llbracket \forall^B D \cdot P \rrbracket^B := \langle \bigwedge_{x \in \mathcal{D}}^z \mathcal{P} x, \text{bool}, \text{pf}_{\forall^B D \cdot P}^{\text{bool}} \rangle \quad \text{where} \quad \text{pf}_{\forall^B D \cdot P}^{\text{bool}}: \bigwedge_{x \in \mathcal{D}}^z \mathcal{P} x \in \llbracket \text{bool} \rrbracket^z$$

Denotational semantics of B

Forall quantifier

We are not done yet!

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^B D.P}^{\text{bool}}: \bigwedge_{x \in \mathcal{D}}^z \mathcal{P} \, x \in \llbracket \text{bool} \rrbracket^z$$

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^B}^{\text{bool}} D.P : \left(\bigwedge^z \{ \mathcal{P} \, x, x \in \mathcal{D} \} \right) \in \llbracket \text{bool} \rrbracket^z$$

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^{\mathbb{B}} D.P}^{\text{bool}}: \left(\bigwedge^z \{ \mathcal{P} \text{ } x, x \in \mathcal{D} \} \right) \in \mathbb{B}$$

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^{\mathbb{B}} D.P}^{\text{bool}}: \left(\bigwedge^z \{y \in \mathbb{B} \mid \exists x \in \mathcal{D}, y = \mathcal{P} x\} \right) \in \mathbb{B}$$

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^{\mathbb{B}} D.P}^{\text{bool}}: \left(\bigwedge^z \{y \in \mathbb{B} \mid \exists x \in \mathcal{D}, y = \mathcal{P} x\} \right) \in \mathbb{B}$$

$$\forall z \in \llbracket \tau \rrbracket^z, \exists (x_i)_{1 \leq i \leq n}, z = (x_1, \dots, x_n)^z \rightarrow \forall i, x_i \in \llbracket \tau_i \rrbracket^z \quad (\text{pf}_{x_i}^{\tau_i})$$

Denotational semantics of B

Forall quantifier

We are not done yet! The following facts remain to be proved:

$$\text{pf}_{\forall^{\mathbb{B}} D.P}^{\text{bool}}: \left(\bigwedge^z \{y \in \mathbb{B} \mid \exists x \in \mathcal{D}, y = \mathcal{P} x\} \right) \in \mathbb{B}$$

$$\forall z \in \llbracket \tau \rrbracket^z, \exists (x_i)_{1 \leq i \leq n}, z = (x_1, \dots, x_n)^z \rightarrow \forall i, x_i \in \llbracket \tau_i \rrbracket^z \quad (\text{pf}_{x_i}^{\tau_i})$$

(easy proof)

Roadmap

- ☒ Syntax
- ☒ Type system
- ☐ Semantics

Roadmap

- ✓ Syntax
- ✓ Type system
- ✓ Semantics

- 1 Introduction
- 2 Encoding B POs in SMT-LIB using HO
- 3 Results
- 4 Denotational semantics of B**
 - Definitions
 - Reasoning about B terms
- 5 Soundness of the encoding

Denotational semantics of B

We can now state **properties** about B terms!

Denotational semantics of B

We can now state **properties** about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$. Then,

$$\llbracket t \rrbracket^B = \langle T, \sigma, \text{pf}_T^\sigma \rangle \rightarrow \sigma = \tau$$

Denotational semantics of B

We can now state **properties** about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$. Then,

$$\llbracket t \rrbracket^B = \langle T, \sigma, \text{pf}_T^\sigma \rangle \rightarrow \sigma = \tau$$

Theorem (Partial correctness of the simplifier)

Assume that $\Gamma \vdash t : \tau$. Let Δ be a renaming context. Then,

$$\llbracket (\mid t \mid \Delta) \rrbracket^B = \langle T, \tau, \text{pf}_T^\tau \rangle \rightarrow \llbracket (\mid \text{simplifier}(t) \mid \Delta) \rrbracket^B = \langle T, \tau, \text{pf}_T^\tau \rangle$$

Denotational semantics of B

We can now state **properties** about B terms!

Theorem (Type correctness of the denotation)

Assume that $\Gamma \vdash t : \tau$. Then,

$$\llbracket t \rrbracket^B = \langle T, \sigma, \text{pf}_T^\sigma \rangle \rightarrow \sigma = \tau$$

Theorem (Partial correctness of the simplifier)

Assume that $\Gamma \vdash t : \tau$. Let Δ be a renaming context. Then,

$$\llbracket (\mid t \mid \Delta) \rrbracket^B = \langle T, \tau, \text{pf}_T^\tau \rangle \rightarrow \llbracket (\mid \text{simplifier}(t) \mid \Delta) \rrbracket^B = \langle T, \tau, \text{pf}_T^\tau \rangle$$

Remark

■ Total correctness does not hold.

- 1 Introduction
- 2 Encoding B POs in SMT-LIB using HO
- 3 Results
- 4 Denotational semantics of B
- 5 Soundness of the encoding**

What we are after

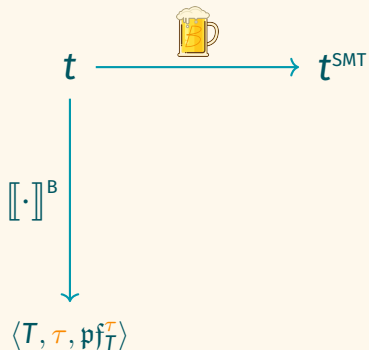
What we are after

t

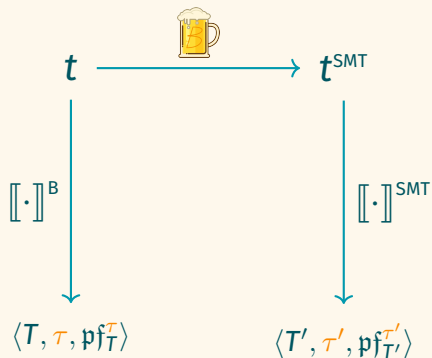
What we are after



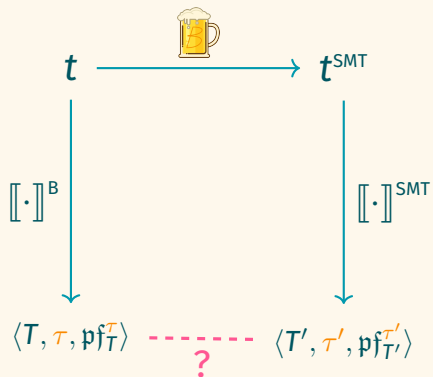
What we are after



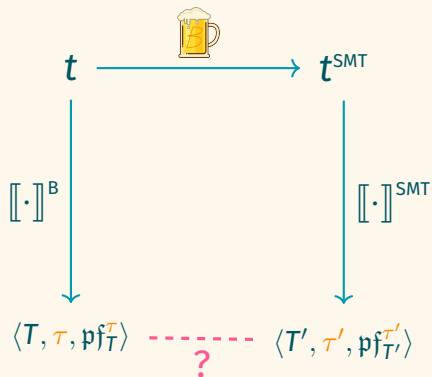
What we are after



What we are after



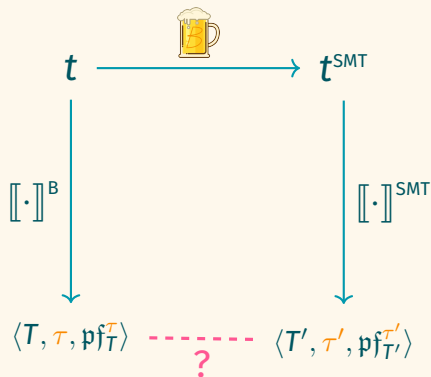
What we are after



We should at least have:

$$\tau' = \tau^{\text{SMT}}$$

What we are after



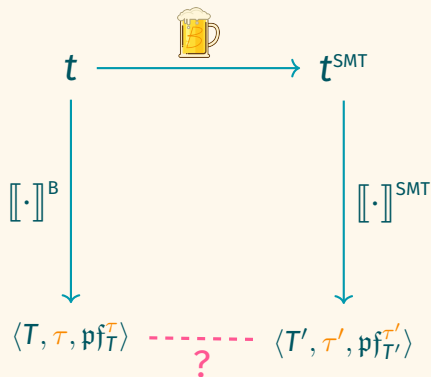
We should at least have:

$$\tau' = \tau^{\text{SMT}}$$

and something like:

T “corresponds to” T'

What we are after



We should at least have:

$$\tau' = \tau^{\text{SMT}}$$

and something like:

T “corresponds to” T'

which has to be formalized.

Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α :

$$\llbracket \alpha \rrbracket^Z \cong \llbracket \alpha^{\text{SMT}} \rrbracket^Z$$

Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α :

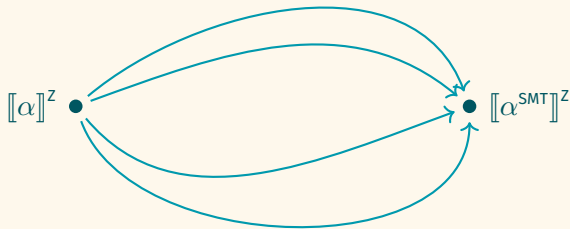
$$\llbracket \alpha \rrbracket^Z \cong \llbracket \alpha^{\text{SMT}} \rrbracket^Z$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any B type α :

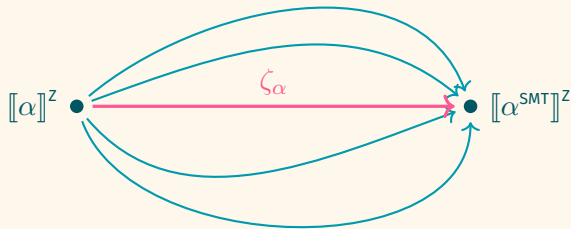
$$\llbracket \alpha \rrbracket^Z \cong \llbracket \alpha^{\text{SMT}} \rrbracket^Z$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any *B* type α :

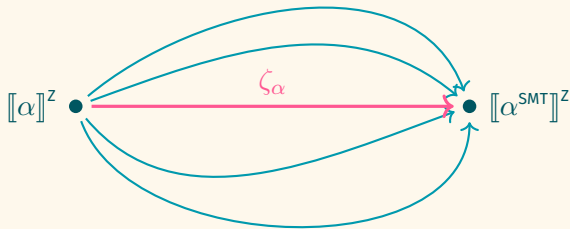
$$\llbracket \alpha \rrbracket^Z \cong \llbracket \alpha^{\text{SMT}} \rrbracket^Z$$



Theorem

B types and their SMT-LIB translations are isomorphic, i.e., for any *B* type α :

$$\llbracket \alpha \rrbracket^Z \cong \llbracket \alpha^{\text{SMT}} \rrbracket^Z$$



Inductively-defined indexed family of **canonical isomorphisms** $(\zeta_\alpha)_{\alpha: \text{BType}}$.

Why care about canonical isomorphisms?

ZFC interpretations of B types are not disjoint in general:

Why care about canonical isomorphisms?

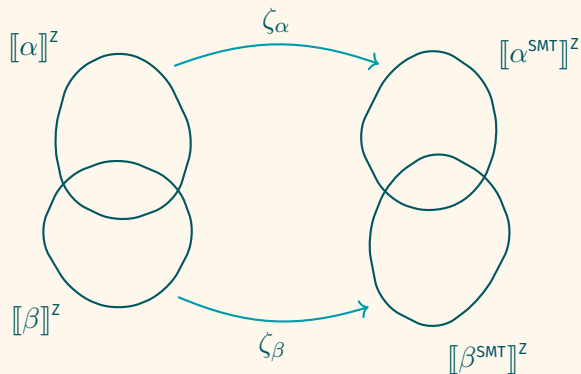
ZFC interpretations of B types are not disjoint in general:

Example

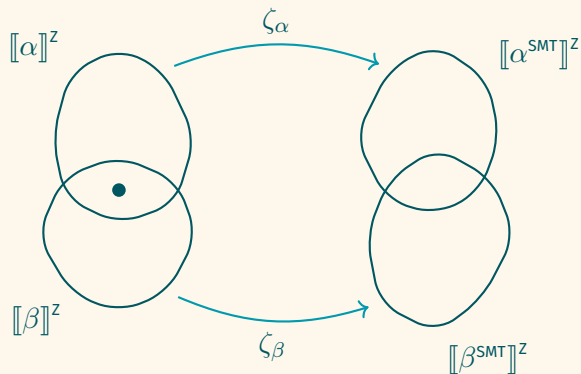
$\llbracket \text{int} \rrbracket^{\mathbb{Z}} = \mathbb{Z}$ and $\llbracket \text{set bool} \rrbracket^{\mathbb{Z}} = \mathcal{P}(\mathbb{B})$ both contain $\{\{\emptyset\}\}$:

$$0_{\mathbb{Z}} = (0_{\mathbb{N}}, 0_{\mathbb{N}}) = \{\{0_{\mathbb{N}}\}, \{0_{\mathbb{N}}, 0_{\mathbb{N}}\}\} = \{\{\emptyset\}\} = \{\top_{\mathbb{B}}\}$$

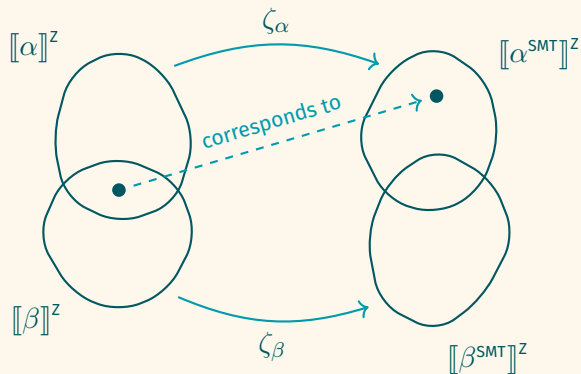
Why care about canonical isomorphisms?



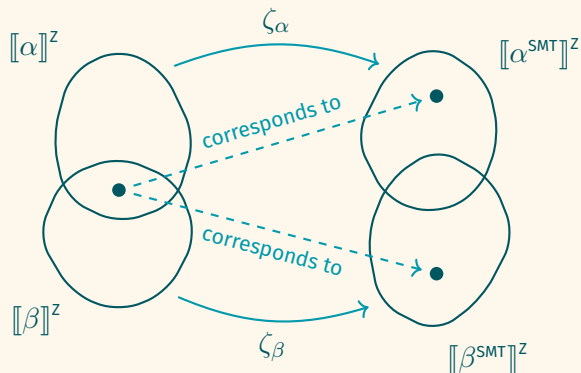
Why care about canonical isomorphisms?



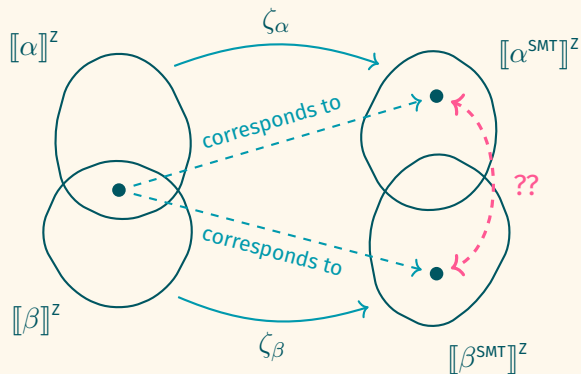
Why care about canonical isomorphisms?



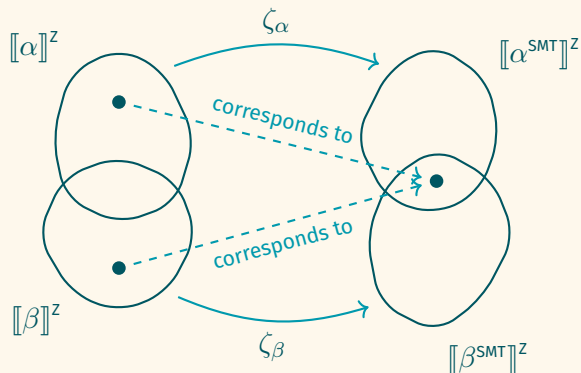
Why care about canonical isomorphisms?



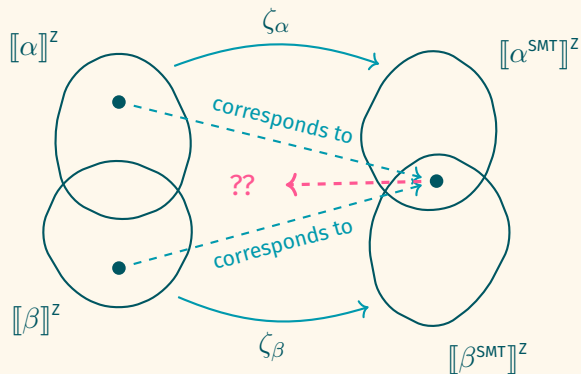
Why care about canonical isomorphisms?



Why care about canonical isomorphisms?

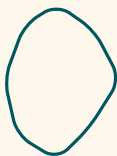


Why care about canonical isomorphisms?

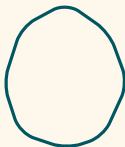


Why care about canonical isomorphisms?

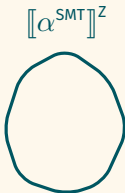
$\llbracket \alpha \rrbracket^Z$



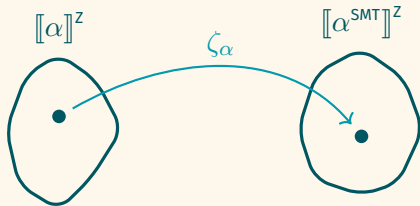
$\llbracket \alpha^{\text{SMT}} \rrbracket^Z$



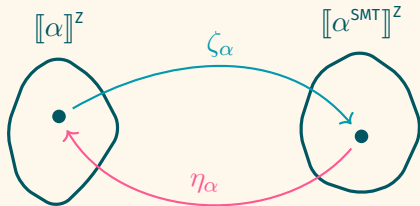
Why care about canonical isomorphisms?



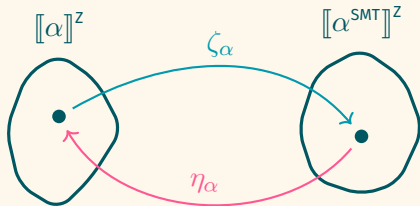
Why care about canonical isomorphisms?



Why care about canonical isomorphisms?

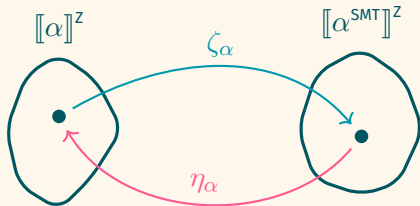


Why care about canonical isomorphisms?



Inductively-defined indexed family of **retractions** $(\eta_\alpha)_\alpha: \text{BType}$

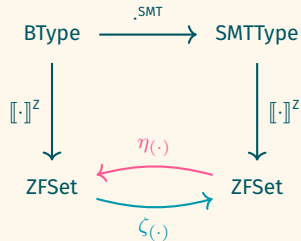
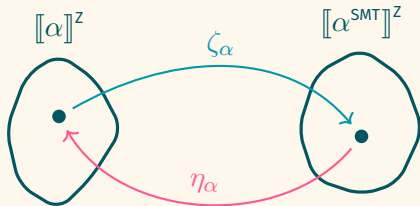
Why care about canonical isomorphisms?



Inductively-defined indexed family of **retractions** $(\eta_\alpha)_\alpha: \text{BType}$

$$\eta_\alpha \circ \zeta_\alpha = \mathbb{1}_{[[\alpha]]^Z}$$

Why care about canonical isomorphisms?

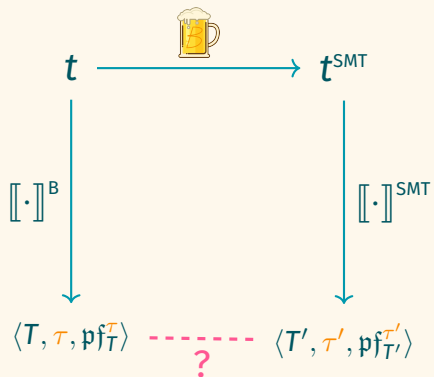


Inductively-defined indexed family of **retractions** $(\eta_\alpha)_\alpha: \text{BType}$

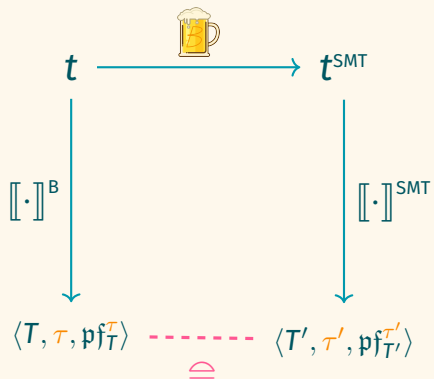
$$\eta_\alpha \circ \zeta_\alpha = \mathbb{1}_{\llbracket \alpha \rrbracket^Z}$$

Yes, η_α is ζ_α^{-1} but we define it constructively as well.

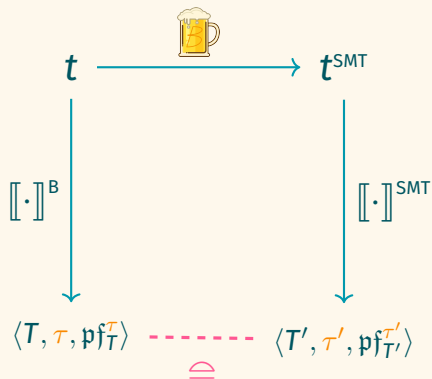
~~What we are after~~ Found it!



~~What we are after~~ Found it!



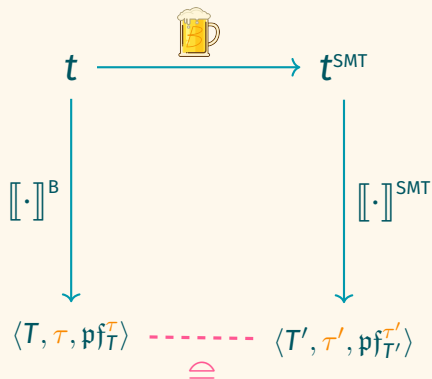
~~What we are after~~ Found it!



We define $\langle T, \tau, \text{pf}_T^\tau \rangle \doteq \langle T', \tau', \text{pf}_{T'}^{\tau'} \rangle$ as:

$$\tau' = \tau^{\text{SMT}} \wedge \eta_\tau(T') = T$$

~~What we are after~~ Found it!



We define $\langle T, \tau, \text{pf}_T^\tau \rangle \doteq \langle T', \tau', \text{pf}_{T'}^{\tau'} \rangle$ as:

$$\tau' = \tau^{\text{SMT}} \wedge \eta_\tau(T') = T$$

Now prove that this is preserved!

Conclusion

Contributions:

- **Higher-order encoding** leveraging SMT-LIB's latest features
- **Formal semantics** for subsets of B proof obligations and SMT-LIB
- **Practical framework** for set-theoretic development in Lean

Current/future work:

- **Correctness** of the encoding



VTreLat/BEer
VTreLat/ZFLean

